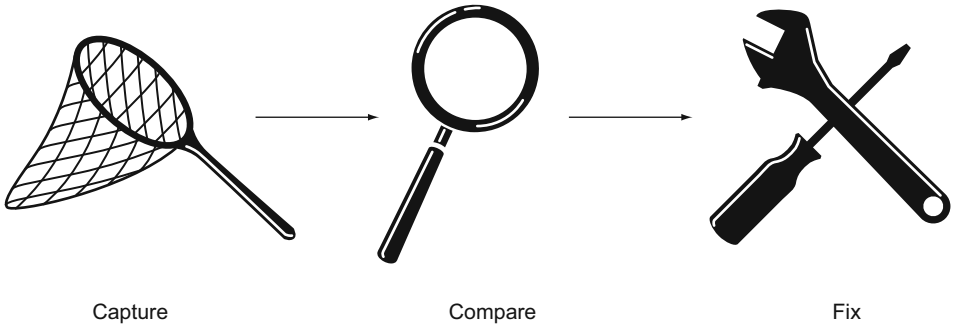


Figure 7.1 Three steps to fixing CORS errors

and test-cors.org to make CORS requests. It ends with resources where you can find answers to your CORS questions.

7.1 Solving CORS errors

With all the platforms, libraries, and frameworks out there, it can be daunting to know where to start debugging CORS requests. But regardless of whether you're using ASP.NET running on Mono or Tomcat running on Linux, debugging CORS comes down to one thing: isolating and analyzing the HTTP headers. The server and client communicate using HTTP headers, and most CORS errors indicate a mismatch between the client request headers and the server response headers.

Regardless of what tools you use to solve the issue, the steps to solving CORS errors are the same (see figure 7.1):

1. Capture a snapshot of the request and response headers
2. Compare the headers to see if and where there is a mismatch
3. Fix the issue by either
   a. Updating the client to send the correct headers
   b. Updating the server to allow the client headers

The rest of this chapter will explore tools you can use to capture and view the HTTP headers. But first, let's create a request you can test against. To understand how to debug broken requests, you need to break a request. The code from chapter 6 introduced configuration options that can be used to configure how a CORS-enabled server behaves. The following listing modifies the configuration options in a way that breaks the client.

Listing 7.1 Configuring the CORS server to only allow the Timezone-Offset header

```
var originWhitelist = [
  'null',
  'http://localhost:1111'
];
```